

中期小结实验报告

双截龙横版动作游戏

C++ / Qt / MVVM 架构设计与实现

课程名称	C++ 项目管理及工程实践
小组名称	双截龙组
小组成员	陈棋隆 (View 层)、邱安琪 (ViewModel 层)、 苏易文 (App/Common 层)
完成日期	2026 年 7 月 13 日

目录

1 实验概述	1
1.1 评分要求理解	1
1.2 组内分工	1
1.3 当前完成情况	2
1.4 报告组织方式	2
2 总体架构设计	3
2.1 架构目标	3
2.2 依赖方向	3
2.3 数据流设计	4
2.4 时间推进机制	4
2.5 状态快照边界	5
3 App/Common 层分报告	6
3.1 层次定位	6
3.2 Common 层设计	6
3.3 App 层设计	8
3.4 关键实现细节	8
3.5 小结	8
4 View 层分报告	9
4.1 层次定位	9
4.2 主窗口与绑定	9
4.3 输入处理	9
4.4 Tick 与游戏循环	10
4.5 绘制流程	10
4.6 世界坐标到屏幕坐标	10
4.7 角色与 HUD 绘制	11
4.8 背景与交互表现	11
4.9 小结	11
5 ViewModel 层分报告	12
5.1 层次定位	12
5.2 GameViewModel 门面	12
5.3 游戏阶段状态机	12
5.4 移动与跳跃	13

5.5 攻击、碰撞与伤害	13
5.6 敌人 AI 与 Boss 战	14
5.7 遭遇战和关卡推进	14
5.8 快照维护	14
5.9 小结	15
6 构建与总结	16
6.1 构建方式	16
6.2 架构收获	16
6.3 后续改进方向	16
6.4 总结	16
7 个人反思与收获	17
7.1 A 同学 - View 层	17
7.2 B 同学 - ViewModel 层	17
7.3 C 同学 - App/Common 层	17

1 实验概述

本实验仓库实现的是一款名为 AlleyFist 的横版动作游戏。项目使用 C++17、Qt6、CMake 与 vcpkg 组织工程，当前已经形成了较清晰的 MVVM 分层：common 层提供跨层公共契约，viewmodel 层负责游戏规则和状态推进，view 层负责 Qt 窗口、输入和绘制，app 层作为组合根管理生命周期和对象绑定。

从玩法目标看，玩家扮演一名格斗角色，从关卡左侧向右推进。推进过程中会遭遇普通敌人，系统通过遭遇战锁屏要求玩家清空当前敌人后才能继续前进；关底触发 Boss 战，击败 Boss 后进入胜利结算。游戏同时支持移动、跳跃、轻攻击、重攻击、空中攻击、血量、精力、疲劳、连击、暂停、失败、重开等核心流程。

1.1 评分要求理解

根据题目要求，报告和仓库需要同时体现协作、架构和文档完整性：

要素	分数	本组对应工作
成员协作紧密，且在版本控制系统上都有有效提交	30	按 app/common、view、viewmodel 分层分工，后续通过提交记录证明每位成员参与情况
完整地按照最先进的框架开发	20	采用 MVVM 思路，将输入命令、状态快照、界面绘制和游戏规则解耦
完整的整体报告和分报告	50	本目录将总报告、三份分报告和个人反思模板统一纳入自动构建流程

1.2 组内分工

本组以层次边界作为分工单位，每位成员既负责对应代码，也负责对应分报告：

- A：负责 View 层，分报告为 03_view_layer.md，主要内容包括窗口、画布、输入、HUD 和覆盖层。
- B：负责 ViewModel 层，分报告为 04_viewmodel_layer.md，主要内容包括命令处理、状态推进、AI、战斗和胜负判定。
- C：负责 App/Common 层，分报告为 02_app_common_layer.md，主要内容包括生命周期、组合根、命令、快照和接口合同。
- 汇总者：负责 00_project_overview.md、01_architecture.md 和构建脚本，统

一整体框架、PDF 排版与自动化合并流程。

1.3 当前完成情况

当前仓库已经完成了游戏的基本可运行骨架和核心流程：

- 工程层面：通过 CMake 分 target 组织 common、viewmodel、view、app 和最终可执行文件 AlleyFist。
- 公共契约：使用 GameCommand 表达输入和 Tick，使用 GameSnapshot 表达完整帧状态，使用 IGameCommandSink 与 IGameSnapshotSource 连接 View 和 ViewModel。
- ViewModel：实现标题、游玩、遭遇战锁屏、清屏可前进、暂停、失败、胜利等阶段，并支持移动、跳跃、攻击、精力恢复、敌人追击、Boss 战和结算统计。
- View：实现 60 FPS 定时器、键盘输入映射、世界坐标投影、街道背景、视差建筑、角色绘制、血条、精力条、Boss 血条、GO 提示和胜负覆盖层。
- App：通过 GameApplication 创建 Qt 应用、主窗口和 ViewModel，并在主窗口中完成公共接口绑定。

1.4 报告组织方式

为了降低最终汇总成本，本报告采用“分文件维护、脚本合并、LaTeX 排版”的方式：

```
report/  
  Markdown/          # 总报告正文与三份分报告  
  latex/             # 封面、字体、页眉页脚和代码块样式  
  build_report.sh    # 自动合并 Markdown 并生成 PDF  
  output/pdf/        # 最终 PDF 输出
```

这样做的好处是，每位成员只需要维护自己负责的 Markdown 文件；汇总者可以直接运行脚本生成最终 PDF，避免手工复制时出现格式不统一、章节顺序错乱或遗漏内容的问题。

2 总体架构设计

2.1 架构目标

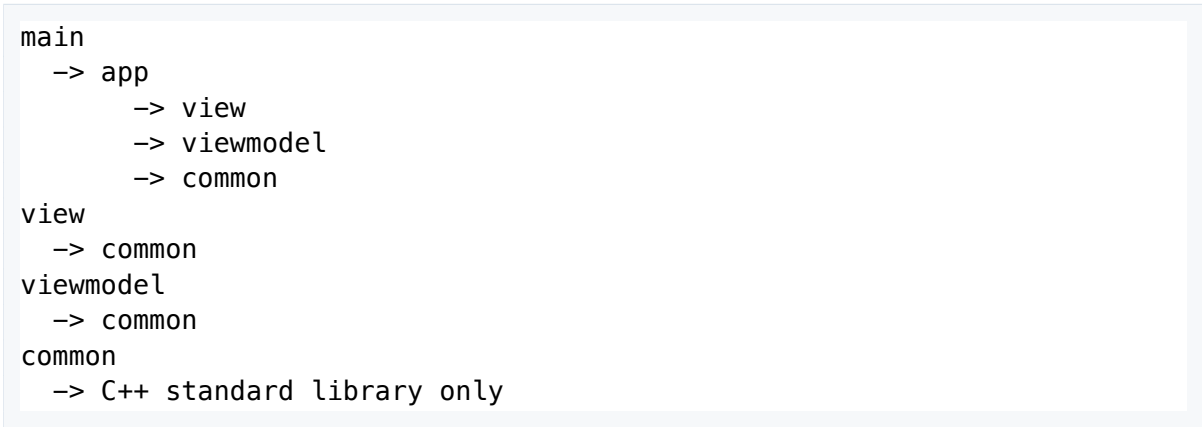
本项目选择 MVVM 风格的分层设计，主要目标有三个：

- 1. View 只处理“如何显示”和“如何接收输入”，不保存玩法规则状态。
- 2. ViewModel 只处理“游戏如何推进”和“当前状态是什么”，不依赖 Qt 控件或绘图 API。
- 3. Common 层只定义两边共享的数据形状和接口，让 View 与 ViewModel 可以通过稳定合同通信。

这个设计使得界面、规则和应用启动逻辑分别处在不同 target 中，后续无论是替换绘制方式、扩展敌人 AI，还是调整应用启动流程，都不需要牵动整个项目。

2.2 依赖方向

项目的依赖方向如下：



app 层可以看到具体的 View 和 ViewModel，因为它承担组合根职责；但 view 和 viewModel 之间没有直接依赖。两者只通过 common/contracts.h 中的接口互相配合。

层次	主要文件	对外暴露	不应承担的职责
Common	actions.h、 snapshot.h、 types.h、 contracts.h	命令、快照、基础值 类型和绑定接口	Qt 绘制、AI、碰撞、 生命周期管理
ViewModel	GameViewModel、 GameSimulation	IGameCommandSink、 IGameSnapshot- Source	直接绘制画面、读取 键盘码

层次	主要文件	对外暴露	不应承担的职责
View	MainWindow、 GameWidget	bind(...)、Qt 窗口 与画布	修改游戏内部状态、 计算伤害
App	GameApplication	run()	游戏规则、具体绘制 细节

2.3 数据流设计

项目中存在两条单向数据流。

第一条是输入通道：

```
Qt key event
-> GameWidget::keyPressEvent / keyReleaseEvent
-> GameCommand
-> IGameCommandSink::handle_command
-> GameViewModel
-> GameSimulation
```

View 将键盘事件翻译成 InputAction 和 ButtonState，例如 MoveRight + Pressed 或 HeavyAttack + Triggered。ViewModel 不需要知道用户按的是方向键、WASD、J 还是 Z，它只处理逻辑动作。

第二条是状态通道：

```
GameSimulation
-> GameSnapshot
-> IGameSnapshotSource::snapshot
-> GameWidget::updateSnapshot
-> paintEvent
```

ViewModel 每帧生成完整的 GameSnapshot。View 只读该快照，并根据其中的 phase、map、player、enemies、hud、result 等字段绘制画面。

2.4 时间推进机制

游戏循环由 View 层的 QTimer 驱动，约每 16 ms 触发一次。触发时，GameWidget 根据 QElapsedTimer 计算真实的 delta time，并发出 tickRequested(deltaSeconds, frameIndex)。绑定器把 Tick 转成 GameCommand::tick_command 后交给 ViewModel。

这种设计兼顾了 Qt 应用的事件循环和 ViewModel 的独立性。定时器本身留在 View 层，规则推进留在 ViewModel 层；窗口只提供时间节奏，不直接参与移动、攻击或胜负判定。

2.5 状态快照边界

GameSnapshot 是 ViewModel 到 View 的唯一状态出口。它不是内部对象本身，而是某一帧的只读显示结果。快照中包含：

- GamePhase：标题、游玩、锁屏、暂停、失败、胜利等阶段。
- MapSnapshot：世界宽度、视口大小、镜头位置、街道边界、GO 提示。
- ActorSnapshot：玩家、敌人、Boss、特效的可绘制状态。
- HudSnapshot：玩家血量、精力、Boss 血条、连击和疲劳状态。
- GameResultSnapshot：胜负原因、用时和击败数量。

内部的攻击盒、碰撞矩形、敌人攻击冷却、遭遇战 ID、滚动锁状态等都保留在 View-Model 内部，不向 View 暴露。这使得 View 可以保持稳定，即使后续增加更复杂的 AI 或连招系统，也只需要在快照层补充必要显示字段。

3 App/Common 层分报告

负责人：C 学号：待填写

3.1 层次定位

App/Common 这一部分承担两个不同但相关的职责。

common/ 是 View 与 ViewModel 之间的公共合同。它不依赖 Qt，也不写具体玩法，只定义两层都能理解的数据结构和接口。app/ 是应用组合根，负责创建 Qt 应用对象、主窗口和 ViewModel，并把 View 与 ViewModel 通过 Common 接口绑定起来。

这两层的共同目标是控制依赖边界：公共协议放在 Common，具体对象创建放在 App，避免 View 和 ViewModel 彼此直接包含对方头文件。

3.2 Common 层设计

Common 层主要由四类文件组成：

文件	作用
types.h	定义 Size、WorldPosition、ResourceBar 等基础值类型
actions.h	定义 InputAction、ButtonState、GameCommand，作为输入通道协议
snapshot.h	定义 GameSnapshot 及其子结构，作为状态通道协议
contracts.h	定义 IGameCommandSink、IGameSnapshotSource 和回调绑定类型

3.2.1 输入命令

输入命令将 Qt 键盘事件抽象为游戏动作：

```
enum class InputAction {
    MoveLeft,
    MoveRight,
    MoveUp,
    MoveDown,
    LightAttack,
    HeavyAttack,
    Jump,
    Restart,
    Confirm,
    Pause
}
```

```
};
```

ButtonState 区分持续输入和瞬时输入。移动类命令使用 Pressed / Released，攻击、跳跃、暂停、确认等命令使用 Triggered。这样 ViewModel 可以自然地区分“正在移动”和“执行一次动作”。

GameCommand 再把输入和 Tick 统一包装起来。View 每帧发出的时间推进也走同一个命令入口，从而让 ViewModel 对外只有一个 handle_command 方法。

3.2.2 状态快照

snapshot.h 将可显示状态拆成几个层次：

- ActorSnapshot 描述单个角色或对象，包括阵营、种类、位置、尺寸、血量、精力、状态、朝向和可见性。
- MapSnapshot 描述地图和镜头，包括世界宽度、视口大小、街道边界、锁屏边界和 GO 提示。
- HudSnapshot 整理 HUD 专用数据，包括玩家血量、精力、Boss 血条、连击和疲劳状态。
- GameResultSnapshot 记录结算数据，包括胜负原因、用时和击败数量。
- GameSnapshot 汇总完整帧状态，作为 View 绘制一帧画面的全部输入。

这样的设计避免 View 从多个内部对象中拼接信息。View 只需要读取快照，ViewModel 负责把内部规则状态整理成适合显示的数据。

3.2.3 绑定接口

contracts.h 是 MVVM 边界的核心：

```
class IGameCommandSink {
public:
    virtual ~IGameCommandSink() = default;
    virtual void handle_command(const GameCommand& command) = 0;
};

class IGameSnapshotSource {
public:
    virtual ~IGameSnapshotSource() = default;
    virtual const GameSnapshot& snapshot() const = 0;
    virtual BindingCookie add_change_callback(ChangeCallback callback) =
        0;
    virtual void remove_change_callback(BindingCookie cookie) = 0;
};
```

View 只依赖这两个接口：输入发给 IGameCommandSink，显示状态从 IGameSnapshotSource 获取。ViewModel 实现这两个接口，但不需要知道具体的 GameWidget。

3.3 App 层设计

GameApplication 是 App 层对外暴露的生命周期接口。它使用 PImpl 隐藏 Qt、主窗口和 ViewModel 的具体类型，公共头文件中只保留构造、析构和 run()。

实现层中，GameApplication::Impl 创建三个核心对象：

```
QApplication qtApp;  
GameViewModel viewModel;  
MainWindow mainWindow;
```

随后调用：

```
mainWindow.bind(viewModel, viewModel);
```

这里第一个 viewModel 作为 IGameCommandSink 接收命令，第二个 viewModel 作为 IGameSnapshotSource 提供快照。App 层知道具体类型，但绑定仍通过公共接口完成，没有让 View 直接依赖 ViewModel 的类定义。

3.4 关键实现细节

3.4.1 PImpl 控制公共接口

GameApplication.h 没有包含 Qt 头文件，也没有暴露 MainWindow 或 GameViewModel。这让 App 的公共接口保持轻量，减少上层包含成本，也避免外部代码绕过组合根直接访问内部对象。

3.4.2 Common 不泄漏模拟细节

Common 只放 View 需要理解的字段。例如 WorldPosition 包含 x、laneY 和 z，因为 View 需要用这些值把角色投影到屏幕上；而攻击盒、敌人冷却、遭遇战 ID 和滚动锁枚举都没有放进 Common，而是保留在 ViewModel 的 SimulationTypes.h 和 GameSimulation 中。

3.4.3 App 不承担业务逻辑

App 层只负责启动和绑定，不处理键盘、不绘制画面、不计算攻击伤害。这个边界使后续扩展时不会把初始化代码和业务代码混在一起。

3.5 小结

App/Common 层的核心贡献不在于复杂算法，而在于建立稳定边界。Common 把命令和快照抽象成跨层协议，App 把具体对象创建集中在组合根中。这样整个项目的依赖方向保持清晰，也为 View 和 ViewModel 分别开发提供了基础。

4 View 层分报告

负责人: A 学号: 待填写

4.1 层次定位

View 层负责“玩家看到什么”和“玩家输入了什么”。它包含主窗口 `MainWindow` 和游戏画布 `GameWidget`，主要使用 Qt 的窗口、定时器、键盘事件和 `QPainter` 完成界面工作。

按照 MVVM 分工，View 层只依赖 Common 层的 `GameCommand` 和 `GameSnapshot`，不直接包含 `GameViewModel`，也不修改玩家、敌人、Boss、地图等内部状态。所有游戏规则结果都由 `ViewModel` 通过快照提供。

4.2 主窗口与绑定

`MainWindow` 继承自 `QMainWindow`，内部创建 `GameWidget` 作为中心控件，并提供：

```
void bind(IGameCommandSink& commandSink,
          IGameSnapshotSource& snapshotSource);
```

实际绑定由 `GameWidgetBinding` 完成。它做了三件事：

- 1. 将 `GameWidget::commandGenerated` 转发给 `IGameCommandSink::handle_command`。
- 2. 将 `GameWidget::tickRequested` 包装成 `GameCommand::tick_command` 后转发给 `ViewModel`。
- 3. 向 `IGameSnapshotSource` 注册变化回调，收到通知后调用 `GameWidget::updateSnapshot` 并触发重绘。

绑定器析构时会通过 `BindingCookie` 注销回调，避免 View 销毁后 `ViewModel` 仍然调用旧回调。

4.3 输入处理

View 层将具体键盘码映射为 Common 层定义的逻辑动作：

按键	逻辑动作
方向键 / WASD	上下左右移动
J / Z	轻攻击
K / X	重攻击
Space	跳跃

按键	逻辑动作
R	重新开始
Enter	确认 / 开始
P / Esc	暂停

移动键在按下时发送 `Pressed`，松开时发送 `Released`，这样 `ViewModel` 可以维护持续移动状态。攻击、跳跃、确认、暂停等操作使用 `Triggered`，表示一次性动作。

为了避免按键自动重复导致多次触发，`GameWidget` 使用 `m_triggeredThisPress` 记录本次按下已经触发过的单次动作；只有按键释放后才允许下一次触发。这保证了攻击、跳跃、暂停等操作的输入语义更加稳定。

4.4 Tick 与游戏循环

`GameWidget` 内部使用 `QTimer` 以约 60 FPS 的频率触发更新。每次触发时通过 `QElapsedTimer` 计算距离上一帧的真实时间间隔，并将过大的 `delta time` 限制到 0.1 秒以内，避免调试断点或系统卡顿后一次性推进过多逻辑。

`Tick` 不在 `View` 层直接修改状态，而是通过 `tickRequested(deltaSeconds, frameIndex)` 交给 `ViewModel`。这样 `View` 层只负责提供时间节奏，真正的游戏规则推进仍然保留在 `ViewModel` 中。

4.5 绘制流程

`paintEvent` 是 `View` 层的主绘制入口，整体流程如下：

```
fill black background
if phase == Title:
    drawOverlay
else:
    drawBackground
    drawStreet
    sort actors by laneY/z
    drawActor for player, enemies and effects
    drawHUD
    drawGOIndicator if needed
    drawOverlay for pause/game over/win
```

标题、暂停、失败和胜利界面都通过覆盖层绘制；正常游戏中则先画背景和街道，再按纵深排序绘制角色，最后绘制 HUD 和提示。

4.6 世界坐标到屏幕坐标

`ViewModel` 输出的是世界坐标：

- x 表示关卡横向推进位置。
- laneY 表示街道纵深位置。
- z 表示离地高度，用于跳跃和空中攻击。

View 层通过以下思路转换到屏幕坐标：

```
screenX = (worldX - cameraX) * scaleX;  
screenY = (laneY - z) * scaleY;
```

窗口缩放时，`resizeEvent` 和 `paintEvent` 会根据快照中的标准视口宽高更新 `m_scaleX` 与 `m_scaleY`，让游戏画面可以适应不同窗口尺寸。

4.7 角色与 HUD 绘制

角色绘制使用 `ActorSnapshot` 中的阵营、种类、状态、朝向、尺寸和资源条。View 层根据不同 `ActorState` 绘制站立、行走、攻击、跳跃、空中攻击、受伤、死亡等姿态；根据 `Facing` 左右翻转角色；根据 `Team` 和 `ActorKind` 区分玩家、普通敌人和 Boss 的颜色。

HUD 绘制集中在 `drawHUD` 中，主要包括：

- 玩家血量和精力条。
- 精力耗尽时的疲劳提示。
- 连击计数。
- Boss 出现时的 Boss 血条。
- 屏幕中央消息。
- 底部关卡进度条。

敌人和 Boss 头顶还会绘制小型血条，帮助玩家判断战斗反馈。

4.8 背景与交互表现

当前 View 层实现了夜色背景、远景建筑、街道路面、人行道、路中虚线、GO 闪烁提示、标题渐变文字、暂停遮罩、失败和胜利结算。远景建筑使用视差滚动，位置变化慢于镜头，增强横向推进时的空间感。

这些表现层效果都只依赖快照，不反向改变游戏规则。例如 GO 提示只读取 `map.showGoIndicator`，是否清屏解锁由 `ViewModel` 决定。

4.9 小结

View 层的核心贡献是把 Common 快照转化为可玩的 Qt 界面，同时严格保持自身职责边界。输入被翻译成逻辑命令，时间被翻译成 Tick，状态被翻译成绘制结果。由于 View 不直接操作规则对象，后续 `ViewModel` 继续扩展玩法时，View 只需要响应新增的快照字段或状态枚举即可。

5 ViewModel 层分报告

负责人: B 学号: 待填写

5.1 层次定位

ViewModel 层负责游戏规则和可显示状态的生成。它接收 View 发来的 GameCommand，推进内部模拟，再输出 GameSnapshot 给 View 绘制。它不依赖 Qt，不读取键盘码，也不调用任何绘图函数。

该层分为两个部分：

- GameViewModel: 对外门面，实现 IGameCommandSink 和 IGameSnapshotSource。
- GameSimulation: 内部模拟核心，维护移动、攻击、敌人、遭遇战、胜负判定等规则。

这种拆分让对外绑定逻辑和内部玩法逻辑分开，也避免了窗口代码与移动、攻击、敌人 AI 等规则直接混杂。

5.2 GameViewModel 门面

GameViewModel 的职责比较克制：

1. 接收 GameCommand。
2. 将输入命令转发给 GameSimulation。
3. 将 Tick 命令转化为一次 step。
4. 保存最新 GameSnapshot。
5. 维护变化回调列表，状态变化后通知 View 更新。

add_change_callback 会返回 BindingCookie，remove_change_callback 根据 cookie 移除回调。notify() 复制一份回调列表后逐个调用，减少回调过程中修改列表带来的风险。

5.3 游戏阶段状态机

当前使用 GamePhase 表示游戏大阶段：

阶段	含义
Title	标题界面，等待确认开始
Playing	正常推进和战斗
EncounterLocked	遭遇战锁屏，必须击败敌人

阶段	含义
ClearToGo	清屏后可继续前进，显示 GO 提示
Paused	暂停
GameOver	玩家失败
Win	击败 Boss 后胜利

输入处理中，Confirm 可从标题、失败、胜利进入新游戏，也可从暂停恢复；Pause 可在游玩状态与暂停状态之间切换；Restart 可随时重置到游玩状态。非游玩阶段会阻止移动和攻击逻辑继续推进。

5.4 移动与跳跃

ViewModel 维护四个移动布尔值：m_moveLeft、m_moveRight、m_moveUp、m_moveDown。持续按键通过 Pressed 和 Released 修改这些状态，apply_movement 在每帧根据当前移动状态计算水平位移和街道纵深位移。

横向移动会更新角色朝向，纵向移动会改变 laneY。角色位置会被限制在世界边界和街道上下边界内；遭遇战或 Boss 战锁屏时，还会被限制在当前锁屏区域内，防止玩家绕过战斗直接推进。

跳跃通过 m_jumpActive、m_jumpElapsed 和规则参数 jumpSeconds、jumpHeight 实现。角色的 z 值按正弦曲线变化：

```
z = jumpHeight * sin(pi * progress);
```

这样可以得到上升和下落较自然的抛物线效果。跳跃会消耗精力，精力耗尽时设置疲劳状态。

5.5 攻击、碰撞与伤害

攻击输入不会立即直接修改敌人，而是先设置待处理攻击：

- 轻攻击对应 LightPunch。
- 重攻击对应 HeavyStrike。
- 跳跃时触发攻击对应 JumpKick。

apply_attacks 根据攻击类型设置 CombatBox，再将局部攻击盒转换到世界坐标，与敌人的身体矩形做相交检测。命中后扣除敌人血量，更新敌人状态为 Hurt 或 Dead，并在击败敌人时增加 defeatedEnemies 计数。

攻击同时会更新连击显示和精力消耗。若精力降为 0，则设置 hud.playerExhausted，后续攻击会被限制，直到能量恢复到足够轻攻击为止。

5.6 敌人 AI 与 Boss 战

update_enemies 每帧更新敌人：

- 存活敌人会向玩家横向靠近。
- 敌人会根据玩家 laneY 调整纵深位置，形成简单的追击行为。
- 敌人接近玩家且冷却结束时会攻击玩家。
- Boss 使用更高移动速度和更高伤害，并在 HUD 中显示独立血条。

敌人和攻击冷却数组保持同序更新。死亡、不可见或血量为 0 的敌人会被过滤出当前敌人列表。

5.7 遭遇战和关卡推进

关卡推进使用触发点和滚动锁控制：

- 玩家到达 kGruntTriggerX 后触发普通敌人遭遇战，生成 3 个小怪。
- 玩家到达 kBossTriggerX 后触发 Boss 战，生成 Boss。
- 遭遇战开始时，m_scrollLock 切换为锁定状态，并设置地图左右边界。
- 场上敌人全部被击败后，普通遭遇战切换为 ClearToGo 并显示 GO 提示。
- Boss 被击败后，切换到 Win，记录胜利原因和用时。

View 只看到阶段、边界、GO 提示和敌人快照，不需要知道内部的遭遇战 ID 或滚动锁枚举。

5.8 快照维护

每帧 step 会更新：

- frameIndex 和 elapsedSeconds。
- 玩家位置、状态、血量、精力。
- 敌人列表和 Boss 状态。
- HUD 中的血量、精力、Boss 血条、连击、疲劳。
- 地图镜头和推进比例。
- 胜负结算数据。

update_camera 根据玩家位置计算镜头目标，并限制在世界边界内。锁屏时，镜头也会受到遭遇战区域约束。update_progress 根据玩家在世界中的横向位置计算进度比例，并扫描敌人列表决定是否显示 Boss 血条。

5.9 小结

ViewModel 层的核心贡献是把游戏规则集中在独立的模拟器中，并把结果整理成 View 可直接使用的快照。当前已经实现横版动作游戏的主要骨架：移动、跳跃、攻击、精力、敌人追击、遭遇战锁屏、Boss 战和胜负结算。后续可以继续扩展更完整的连招、击飞、敌人包抄和关卡配置表，而不需要改变 View 与 ViewModel 的基本通信方式。

6 构建与总结

6.1 构建方式

项目使用 CMake Presets 与 vcpkg 管理依赖。首次配置环境时需要准备 vcpkg，并设置 VCPKG_ROOT。配置和编译命令如下：

```
cmake --preset vcpkg
cmake --build --preset vcpkg
```

报告本身的构建命令如下：

```
cd report
./build_report.sh
```

脚本会自动读取 Markdown/*.md，按文件名顺序合并，使用 pandoc 生成 LaTeX，再通过 xelatex 输出最终 PDF。

6.2 架构收获

本实验最重要的收获是通过 MVVM 把“输入、规则、显示”拆开。起初横版动作游戏看起来很容易写成一个大类：键盘事件里直接改坐标，绘制函数里顺手判断攻击，敌人 AI 和窗口状态混在一起。但这样会导致代码很难并行开发，也很难保持层次边界。

当前实现将命令、快照和绑定接口放入 Common 层，使 View 和 ViewModel 可以独立演进。ViewModel 负责规则，View 只根据快照完成绘制。这对三人分工尤其重要：每个人可以围绕自己的层次推进，不必频繁修改其他人的内部实现。

6.3 后续改进方向

当前项目已经具备核心玩法骨架，但仍有一些可以继续完善的方向：

- 将敌人和遭遇战配置从硬编码常量抽离成配置表。
- 增加更完整的连招链、击退、击倒、无敌帧和受击硬直。
- 引入图片、精灵动画和音效资源，替代当前的基础 QPainter 几何绘制。
- 在提交前补齐每位成员的提交记录说明和个人反思。

6.4 总结

本阶段完成了一个可运行、分层边界较清晰的横版动作游戏原型。项目从工程组织、公共协议、界面绘制到规则模拟都已经形成闭环。报告目录也完成了 Markdown 分工和 PDF 自动构建，为最终提交提供了稳定的汇总方式。

7 个人反思与收获

本章按题目要求由每位同学独立完成。下面仅提供统一格式模板，最终提交前请每位成员将占位内容替换为自己的真实反思与收获。

7.1 A 同学 - View 层

姓名：待填写

学号：待填写

负责内容：View 层，包括 MainWindow、GameWidget、键盘输入、绘制和 HUD。

个人反思与收获：

待 A 同学填写。建议结合以下方面展开：Qt 事件循环和绘制机制、如何把键盘事件转换成逻辑命令、如何根据快照绘制界面、遇到的调试问题、对 MVVM 分层的理解变化。

7.2 B 同学 - ViewModel 层

姓名：待填写

学号：待填写

负责内容：ViewModel 层，包括 GameViewModel、GameSimulation、移动、攻击、敌人、遭遇战和 Boss。

个人反思与收获：

待 B 同学填写。建议结合以下方面展开：命令队列、Tick 推进、状态机设计、碰撞和攻击盒、敌人 AI，以及如何避免 ViewModel 依赖具体界面。

7.3 C 同学 - App/Common 层

姓名：待填写

学号：待填写

负责内容：App/Common 层，包括公共类型、命令、快照、绑定接口、应用组合根和生命周期管理。

个人反思与收获：

待 C 同学填写。建议结合以下方面展开：公共契约设计、接口边界、PImpl、组合根、层间依赖控制，以及如何支持组内并行开发。